

Exercise 6

```
[ec2-user@ip-172-31-20-184 lab8]$ mpicc -o pi_anysource pi_anysource.c -lm
mpirun --oversubscribe -np 4 ./pi_anysource
Rank 0: trials = 2500000, count = 1962662
Root received count = 1964200 from rank 3
Root received count = 1962547 from rank 1
Root received count = 1963912 from rank 2

[Pi ANY_SOURCE] Processes = 4
[Pi ANY_SOURCE] Trials = 10000000
[Pi ANY_SOURCE] Points inside = 7853321
[Pi ANY_SOURCE] Estimate of pi = 3.1413284000
[Pi ANY_SOURCE] Error = 0.0002642536
[Pi ANY_SOURCE] Time = 0.265539 sec
[ec2-user@ip-172-31-20-184 lab8]$ |
```

In `pi_anysource.c` the source argument of `MPI_Recv` on root was changed from `r` to `MPI_ANY_SOURCE` (the tag is still 0), and an `MPI_Status` is used to read which rank the message came from (`status.MPI_SOURCE`). Root now takes the counts in the order they arrive instead of rank order: in this run rank 3 arrived first, then rank 1 and rank 2, while the original program always receives 1, 2, 3.

The result is identical to Exercise 3 (7,853,321 points inside, $\pi = 3.1413284$) because the same seeds are used and the order of the additions does not change the total.

Processes	Exercise 3 (source = r)	MPI_ANY_SOURCE
1	351.41 ms	352.58 ms
2	263.77 ms	264.41 ms
3	270.82 ms	269.19 ms
4	273.35 ms	271.87 ms
5	269.84 ms	276.27 ms
6	273.56 ms	274.13 ms
7	270.86 ms	270.49 ms
8	252.59 ms	261.72 ms

The times (median of 10 runs) are almost the same as Exercise 3 for every process count; the largest difference is 3.6 % at 8 processes (261.72 ms against 252.59 ms). Root still has to wait for every count and each process sends only one 8-byte message. MPI_ANY_SOURCE helps when the processes finish at different times and root can use each result as soon as it arrives; here every process has the same amount of work.